CS637. Homework 4. Xiyao long

The is a kubernetes distributed system adoption for the homework3(The movie ticketing system). In my homework 3, each client has its corresponding backend. As there are 5 clients, this make it a lot of 10 running pods in the kubernetes deployment. The original source code for homework 3 is modified and compiled , then made into a image and pushed into Dockerhub. For running this application on kubernetes, I use 10 yaml file each specifying the launching a statefulset on a single pod. A separate file used for each pod is because I attempt to specify a client with a specific index such as 1, this client can be mapped to a backend node with yaml file specifying its running pod with index 1 as well.

Contents:

Source code is in ./src/main/java/com.example
Config file in ./src/main/resources/application.config
The yaml files for launching kubernetes is in ./kubernetes
The application binary is at ./target/app.jar

Image creation (already completed)

```
# create the docker image
docker build -t xlong2/docker_image .

# push the docker image
docker push xlong2/docker_image:latest
```

To run on kubernetes:

```
#1. Start a googlecloud cluster ; make sure kubectl is installed.
#2.
     cd ./kubernetes
#3. launch the application
    kubectl apply -f .
#4. check  the pods
    kubectl get pods
   kubectl logs -f $pod_id
```

Modification of the hw3 ticketing system program:

1. In the main function, it will get the environmental variables of the pod that runs the application. The variables are specified in the yaml file for the pod. If the yaml file has NODE_TYPE variable being “Client” it will make a Client actor in that pod, else if the NODE_TYPE is “Backend_node” it will make a Node actor(which is basically the ticketing system backend). In the main function, we wait for some time before making

of a Node but we do not wait before making a Client actor. This is intended to let clients get notified that pods with Node type actor are up in the actor system. (Figure 1)

2. Added a receive case for the client actors: when the client actor get notified that a member is up on the cluster, it will check whether the new container is running an actor named as its corresponding backend. (Figure 2)

3. Other standard modification are present such as specify that the newly created actors will join the seed nodes. The previous actor selection path is changed from "/user/*" to "aka://ClusterSystem@*:2551/user/*".(Figure 3) And all the original actor classes has a new attribute — the actor system.

Figure 1.

```
private static final String CLUSTER_NAME = "ClusterSystem";

public static void main(String[] args) throws IOException {


   // Config conf = ConfigFactory.load("application");
    ActorSystem actorSystem = ActorSystem.create(CLUSTER_NAME);
    // actorSystem.actorOf(SimpleClusterListener.props());
    final ActorMaterializer materializer = ActorMaterializer.create(actorSystem);
    Cluster cluster = Cluster.get(actorSystem);
    // the seed nodes just sit around and do nothing.
    System.setProperty("NODE_TYPE","Client");
    if(System.getenv().get("NODE_TYPE").equals("Client")){
       // actorSystem.actorOf(SimpleClusterListener.props());
        ActorRef aclient = actorSystem.actorOf(Props.create(Client.class),  name: "frontendNode"+System.getenv().get("CLIENTINDEX"));
       // ServiceDiscovery serviceDiscovery = Discovery.get(actorSystem).discovery();

    }else if(System.getenv().get("NODE_TYPE").equals("Backend_node")){
        try {
            Thread.sleep( millis: 2000);
        }catch (Exception e){
            e.printStackTrace();
        }
        //actorSystem.actorOf(SimpleClusterListener.props());
        actorSystem.actorOf(Props.create(NodeOnFsm.Node.class),  name: "backendNode" +  System.getenv().get("BACKENDINDEX"));

    }

    List<Address> addresses = Arrays.asList(System.getenv().get("SEED_NODES").split( regex: ",")).stream()
            .map(ip -> new Address( protocol: "akka", CLUSTER_NAME, ip,  port: 2551))
            .collect(Collectors.toList());

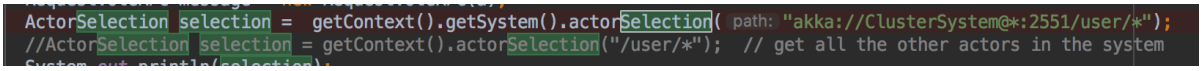
    cluster.joinSeedNodes(addresses);  //This is the last ide
}
```

Figure 2. Inside Client.java

```
@Override
public Receive createReceive() {
    return receiveBuilder()
            .matchEquals( object: "done", m -> getContext().stop(getSelf()))  // this stops the actor ifsel
            .match(ClusterEvent.MemberUp.class,
                    m->{
                        Thread.sleep( millis: 500);  // wait for the backend nodes launch its actors
                       System.out.println("New member joined, the address is "+ m.member().address());
                        String selfid = getselfid();
                        char self_index = selfid.charAt(selfid.length()-1);
                        ActorSelection tentativeBackend = context().actorSelection(  path: m.member().address()+ "/user/backend"+self_index);
                        Timeout t = new Timeout( length: 5, TimeUnit.SECONDS);
                        AskableActorSelection asker = new AskableActorSelection(tentativeBackend);
                        Future<Object> fut = asker.ask(new Identify( messageId: 1), t);
                        ActorIdentity ident = (ActorIdentity)Await.result(fut, t.duration());
                        ActorRef ref = ident.getRef();
                        if(ref!=null){
                            backendNode = context().actorSelection(  path: m.member().address()+ "/user/backend"+self_index);
                        }


                    }
```

Figure 3: the actor selection method is changed to reflect that the actor system is now a cluster.



Kubernetes deployment:

1. All pods communicate through port 2551.
2. The two seed nodes use image built by someone else
3. The other 10 pods use the image build by me. (The main application)

Resources used:

1. The code is Figure 1 is large adopted from: https://stackoverflow.com/questions/18012945/how-can-i-check-if-an-akka-actor-exists-akka-2-2
   This is from the effort in searching a solution to let a specific client identify its corresponding backend.
2. Example project named as “akka-kubernetes-example” in the https://github.com/saturnism/akka-kubernetes-example
3. Kubernetes documentation.

Bugs and Reflection:

In the log of a kubernetes pod:

error message: application.conf @ jar:file:/app/app.jar!/application.conf: 15-35: No configuration setting found for key 'akka.version’

In retrospect; It might be easier to add message mechanism between client and actors to help a client identify who is its corresponding actor. This can be more reliable than the method in Figure 2, which is essentially string matching between actor address of the client and the backend node. The figure 2 method is not tested and the actual actor address need to be double checked.